Entwurf digitaler Systeme

Übung 9

Ziel

- Entwicklung eines Verhaltensmodells einer Standardkomponente
- Entwicklung parametrisierbarer Hardwaremodelle

Vorbereitung

Generics

Alle bislang entwickelten Hardwaremodelle sind hinsichtlich ihres Verhaltens und ihrer Struktur fest definiert. Beispiele fester Definitionen sind

- Zähler mit fest definierter Obergrenze, d. h. maximalem Zählwert
- RAM mit fest definierter Datenwortbreite und Adresstiefe
- Gatter mit fest definierter Verzögerungszeit

```
entity counter is
  port (clk           : in  bit;
        count_value   : out integer range 0 to 15 );
end;

architecture behavior of counter is

  signal int_count_value : integer range 0 to 15;
begin
  process
  begin
    wait until clk = '1';
    if (int_count_value = 15) then
      int_count_value <= 0;
    else
      int_count_value <= int_count_value + 1;
    end if;
  end process;

  count_value <= int_count_value;
end;
```

Beispiel 1: VHDL-Modell eines Zählers mit fest definiertem Zählbereich

Das in Beispiel 1 gezeigte Modell eines Zählers ist nicht änderungsfreundlich, weil die Änderung des Zählbereichs zahlreiche Änderungen im Code zur Folge hätte. Eine änderungsfreundliche Variante des Modells verwendet ein Package, in welchem wesentliche Parameter zentral deklariert werden, siehe Beispiel 2.

Beachten Sie, dass das Modell in Beispiel 2 zwar änderungsfreundlicher ist, die Zählereigenschaften jedoch nach wie vor fest definiert sind. In der Praxis werden häufig gleiche Hardwarekomponenten unterschiedlicher Größen benötigt, beispielsweise gleichartige Zähler mit verschiedenen Zählbereichen. Es gibt zunächst zwei Ansätze, um diese Anforderung zu erfüllen:

- Bereitstellung der Komponenten in allen benötigten Größenvariationen
- Einmalige Erstellung eines Hardwaremodells mit den maximalen Größenparametern und generelle Verwendung dieses Modells

Es ist leicht einzusehen, dass beide Ansätze ineffizient sind. So ist im ersten Fall eine größere Anzahl an Hardwaremodellen zu verwalten. Je mehr VHDL-Code erzeugt wird, desto größer ist der Validierungsaufwand. Im zweiten Fall wird nur ein einzelnes Modell erstellt, jedoch mit den maximal benötigten Parameterwerten. Dies führt überall dort, wo kleinere Komponenten ausreichend wären, zu unnötig hohem Schaltungsaufwand (Chipfläche).

```
package count_pack is
  constant max_cnt : integer := 15;
  constant min_cnt : integer :=  0;
end;


use work.count_pack.all;
entity counter is
  port (clk           : in  bit;
        count_value   : out integer range min_cnt to max_cnt );
end;

architecture behavior of counter is

  signal int_count_value : integer range min_cnt to max_cnt;
begin
  process
  begin
    wait until clk = '1';
    if (int_count_value = max_cnt) then
      int_count_value <= min_cnt;
    else
      int_count_value <= int_count_value + 1;
    end if;
  end process;

  count_value <= int_count_value;
end;
```

Beispiel 2: Flexibleres VHDL-Modell eines Zählers

Als Ausweg bietet VHDL die Möglichkeit, parametrisierbare Hardwaremodelle zu erstellen. Als Parameter wählt man die Größen, die zur Anpassung an die verschiedenen Einsatzfälle nötig sind. Solche Parameter heißen in VHDL *Generic Constants,* kurz *Generics*. Die Deklaration von Generics erfolgt innerhalb einer Entity. Sie sind sichtbar in der Entity sowie allen zugehörigen Architectures.

```
entity counter is
  generic (max_cnt : integer;
           min_cnt : integer);

  port    (clk           : in  bit;
           count_value   : out integer range min_cnt to max_cnt );
end;

architecture behavior of counter is
  signal int_count_value : integer range min_cnt to max_cnt;
begin
  process
  begin
    wait until clk = '1';
    if (int_count_value = max_cnt) then
      int_count_value <= min_cnt;
    else
      int_count_value <= int_count_value + 1;
    end if;
  end process;

  count_value <= int_count_value;
end;
```

Beispiel 3: Parametrisierbares VHDL-Modell eines Zählers

Beachten Sie in Beispiel 3, dass kein Package benötigt wird. Offen ist noch die Frage, wie den Generics konkrete Werte zugewiesen werden. Hierbei unterscheidet man drei Möglichkeiten.

a) Zuweisung aktueller Werte an Generics bei der Komponenten-Instanziierung

Es existiert nur ein einziges Zählermodell, jedoch kann der Zähler mehrfach in verschiedenen Größen in übergeordneten Modellen instanziiert werden, siehe Beispiel 4. Neben der „Port-Map“ wird hierzu jeweils eine „Generic-Map“ mit den entsprechenden Werten angegeben.

```
architecture behavior of counter is
begin
  process
  begin
    c_20_0: counter generic map (20, 0)
                    port map (clk, cnt_val_1);

    c_9_1:  counter generic map (9, 1)
                    port map (clk, cnt_val_2);
end;
```

Beispiel 4: Generic-Map

Die Werte von Generics können durch die Schaltungshierarchie nach unten durchgereicht werden. Die höchste Ebene zur Zuweisung von Werten in einer Generic-Map ist die VHDL-Testbench.

b) Zuweisung aktueller Werte bei der Elaboration der Komponente

Enthält das Hardware-Modell der höchsten Hierarchieebene (z. B. auch die Testbench) Generics, so werden aktuelle Werte im Zuge der Elaboration im VHDL-Simulator angegeben.

c) Zuweisung aktueller Werte durch Angabe von Default-Werten

Eine Generic-Deklaration kann einen Default-Wert enthalten. In diesem Fall kann eine explizite Zuweisung nach a) oder b) entfallen, wenn der Default-Wert gewünscht ist. Ansonsten überschreibt eine explizite Zuweisung den Default-Wert.

```
entity counter is
  generic (max_cnt : integer := 15;
           min_cnt : integer :=  0);
  ...
```

Beispiel 5: Generics mit Default-Werten

Weitere Beispiele zur Anwendung von Generics finden Sie in Kapitel 9 des Skripts.

Modellierung von Standardkomponenten

Die Validierung von Hardwaremodellen durch VHDL-Simulation erfordert eine möglichst realistische Nachbildung der Systemumgebung des zu entwerfenden Bausteins. Die Systemumgebung enthält in der Regel Standardkomponenten wie beispielsweise Mikroprozessoren und Speicher. Moderne hochintegrierte Bausteine vereinen häufig alle diese Komponenten auf einem einzigen Chip (System-on-Chip), wobei Standardkomponenten dann als Makros bezeichnet werden. In beiden Szenarien werden zur Validierung des Gesamtsystems Verhaltensmodelle der Makros benötigt, welche im Normalfall zusammen mit den eigentlichen Makros zur Verfügung gestellt werden.

In der vorliegenden Übung übernehmen Sie die Rolle eines Makroanbieters und entwickeln ein einfaches Verhaltensmodell einer Standardkomponente. Das Verhaltensmodell muss nicht zwangsläufig synthetisierbar sein, da es nur zur Validierung eingesetzt wird. Die Synthese entfällt.

Aufgabenbeschreibung

Beschreibung eines synchronen SRAMs

Es soll eine parametrisierbare Beschreibung eines synchronen SRAM-Bausteins mit getrennten Daten-Ein- und -Ausgängen entworfen werden. Die Anschlüsse des Bausteins sind Abbildung 1 zu entnehmen.

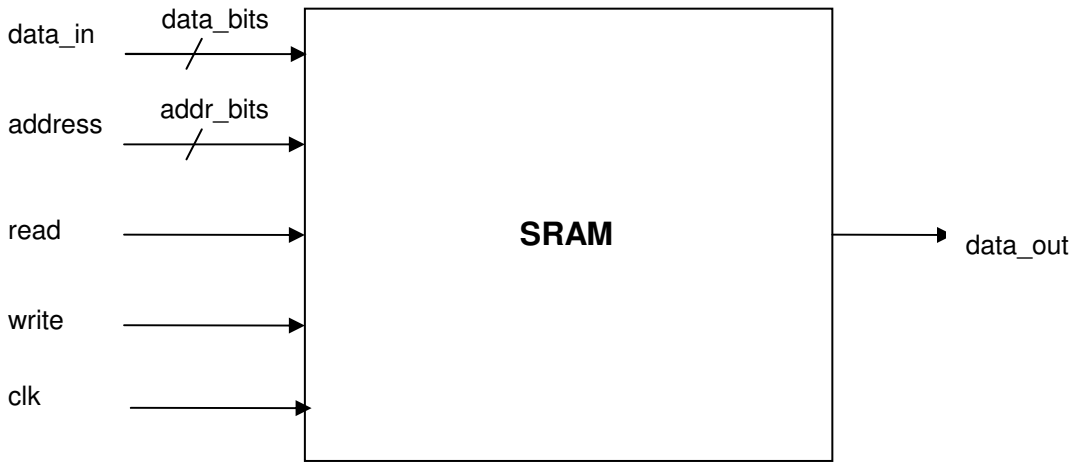


Abbildung 1: Anschlüsse des SRAM-Bausteins

Die Datenports sind vom Typ signed, der Adressport vom Typ unsigned. Die Größen der Ports werden zunächst nach Bedarf im Package ram_pack festgelegt.

Der Datentyp der restlichen Eingangsports ist std_ulogic.

Ergänzen Sie die Entity- und Architecture-Beschreibung in sram.vhd nach folgenden Vorgaben:

- Schreib- und Lesezugriffe auf das RAM erfolgen synchron zur steigenden Flanke des Taktsignals clk.
- Ein Schreibzugriff erfolgt nur, wenn write = ´1´ ist.
- Ein Lesezugriff erfolgt nur, wenn read = ´1´ ist.
- Bei gleichzeitigem Schreib- und Lesezugriff wird der zuletzt gespeicherte Wert gelesen, nicht der neue Wert. Der neu geschriebene Wert ist erst beim nächsten Lesezugriff verfügbar.
- Erfolgt kein Lesezugriff (read = ´0´), so befindet sich das Ausgangssignal im hochohmigen Zustand “ZZ...Z“.
- Auszulesende Daten erscheinen nicht unmittelbar zum Zeitpunkt einer steigenden Taktflanke, sondern um die Zeit τ verzögert am Datenausgang. τ wird ebenfalls im Package ram_pack spezifiziert.

Hinweise:

Der RAM-Inhalt kann als eindimensionale Array-Variable definiert werden. Als Indextyp bietet sich integer bzw. natural (beschränkt auf Anzahl der Adressen) an, Elementtyp ist der Datentyp des Eingangssignals data_in.

Beachten Sie, dass der Adress-Eingang address des RAMs vom Typ unsigned ist, der Indextyp der RAM-Variablen jedoch integer. Sie müssen hierzu eine Anpassung vornehmen.

Zur Validierung der RAM-Beschreibung steht eine Testbench tb_ram.vhd zur Verfügung.

Die Größe der Daten- und Adressports des SRAM-Bausteins wurden zuvor durch Konstantendeklarationen in einem Package fest definiert. Dadurch lässt sich zwar relativ einfach ein SRAM anderer Größe erstellen, jedoch kann jeweils nur eine einzige Größe definiert werden. Es ist davon auszugehen, dass SRAMs in einer Vielzahl verschiedener Konfigurationen bzgl. Bitbreite und Adresstiefe benötigt werden.

Ändern Sie nun die Beschreibung des SRAM-Bausteins, indem Sie die im Package enthaltenen Deklarationen in Generics umsetzen. Vergessen Sie nicht, auch die Testbench anzupassen.

Hinweis: Das Ergebnis dieser Aufgabe wird in einer nachfolgenden Übung benötigt.

Zusatzaufgabe:

In der Praxis werden gleichzeitige Schreib-/Lesezugriffe normalerweise nicht die gleiche RAM-Adresse verwenden. Üblich sind daher RAM-Bausteine mit vollständig getrennten Read- und Write-Ports, d. h. es gibt separate Schreib- und Leseadressen.

- Erweitern Sie das RAM-Modell um eine separate Leseadresse. Nennen Sie die Ports dann

r_addr	Leseadresse
w_addr	Schreibadresse

Ein synchrones SRAM lässt sich grundsätzlich als Register-Array realisieren, bestehend aus einem Feld von Flip-Flops sowie einem Adressdecoder und einem Ausgangsmultiplexer. Eine solche Implementierung erfordert jedoch einen höheren Schaltungsaufwand und wird daher allenfalls für kleine Speicherblöcke gewählt. Für größere Speicher ist es günstiger, spezielle RAM-Technologien zu verwenden, welche einen deutlich geringeren Chipflächenbedarf haben. Nachteil dieser Lösung ist die Notwendigkeit eines speziellen Fertigungsprozesses und damit höheren Kosten.

Wenn Ihr zuvor erstelltes SRAM-Modell synthetisierbar ist, können Sie ein Register-Array generieren. Wählen Sie dafür nicht allzu große Parameterwerte.